IntyBASIC Tips/Tricks and Do's and Don'ts

By Tarzilla
September 18, 2016 in Intellivision Programming

Reply to this topic

Ignore this topic

Report

+Tarzilla
Posted September 18, 2016

I've written more lines of game code and bitmap graphics in the last 5 months than I have in the last 30 years, all because of IntyBASIC, my wife says I'm an idiot, I blame Oscar! 😁 😉

Game programming can be fun, but it can also be hard work and stressful, especially when deadlines are involved. To help others avoid some of the wasted hours of debugging near the end of a project, I thought I'd do a brain dump of some of the things I learned since IntyBASIC came out, but focusing on the last 5 months, specifically with IntyBASIC Showcase Vol 1, but also from other projects. They will be in no specific order and I will add to the first post over the next couple of weeks. Some people may not agree with my thoughts, and sometimes I will dumb down the item (so it may not be 100% technically correct) but feel free to debate below. 👍

Also, there is no intention to insult or berate others, I may give some specific examples but they are not meant to target or insult any specific game or programmer, please don't resort to that in the responses.

(We also don't need to read constant updates of anyone's Ebay auctions, either...)

Latest Version
Make sure you are using the latest version via this Pinned Thread: http://atariage.com/forums/topic/248209-the-intybasic-compiler-official-thread/
There have been so many changes from version to version, all designed to make someone's life easier, whether bug fixes or providing more support for hardware features.

Read the Manual.txt
Or read the Manual_es.txt if you can. The top always lists the changes, especially if new keywords or functions are introduced.

Plan ahead
So many of us are guilty of it. We get a fire in our bellies with a new tool like IntyBASIC and bang out a bunch of code to get MOB's moving or drawing a cool title screen, a lot of time it is before there is an idea, then we keep adding and next thing we have a "game", but then we have a mish-mash of spaghetti code and **GOTO**s. If you think your playing around has given you an idea for a game, then *start over* after you put a plan down on paper, even a rough one.

Good game-making isn't guess work like it was in the 70s and even 80's. Good games have a certain structure (with variations of course.) Write down some rough game loop logic and game notes: The example below is by no means complete or required, but even this forces you to plan ahead, even if you don't end up including things like an Attract mode or High Score Tracking or Saving High Score.

```
Init Game Variables
Show Title Screen
Show Attract Mode
Start Game
Draw Screen(s)
GameLoop
    Check Controllers
    Move Player(s)a
    Move Bad Guys
    Check Collisions
Intermission(s)/ Level complete
GameOver
    Show/Save High Score
```

Use PROCEDURES

Old school BASIC is notorious for GOTO's everywhere, GOSUB PROCEDUREName makes it way easier to slice code into logical chunks that let you do something, then return when done, saving you from having to hand code where to GOTO if you want to get back to where you were. It is especially useful to encapsulate repeatedly called code for reuse and space saving.
Using the stub above now lets you break things into some sort of logical PROCEDUREs

```
Gosub INITGameVariables
Gosub ShowTitleScreen
     Rem Show Attract Mode called from ShowTitleScreen
StartGame:
Gosub DrawScreen
GameLoop:
    Gosub CheckControllers
    Gosub MovePlayer(s)
    Gosub MoveBad Guys
    Gosub CheckCollisions
    If GameOverCondition=1 then gosub ShowGameOver
Goto GameLoop

rem called from ShowGameOver Gosub ShowHighScore
```

Use the CONSTANTS.BAS

The rag-tag group of international scoundrels that cobbled together the IntyBASIC SDK put a lot of work into hiding a lot of the magic numbers, even if you don't use most of them.

SPRITE 0, VISIBLE+Playerx+ZOOMX2, Playery+ZOOMY2 , SPR10+SPR_ORANGE

is infinitely preferable to

SPRITE 0, $0200+Playerx+$400,Playery+$0100, $0850+$1002

Plan to be big, even if you aren't going to be

You may want to try to test yourself by making a game with the limitations like the pioneers (2k/4k even 8k) Drop that idea right now. If you want to do that then delete IntyBASIC and use hand assembler and call routines from the EXEC like they did. IntyBASIC provides a lot of library routines that you don't have control over and as fast and efficient as it has become, you won't be making a game like the 80s just because you end up with an 8k rom. With LTO Flash! available and PCB's available supporting cart sizes well past 16k, forget the old limitations and make a good game.

Use the 42k.bas file included with the latest version. Intentionally stick things into the higher segments that won't change. Title Screen and their bitmaps/screen statements, music statements etc. Save the top segment for your game code as it will be constantly changing, risking the danger of having to reorganize or add ASM ORG statements. Use the 42k.bas template even if your game only ends up being 26.5K, the LTO Flash! or modern cart boards won't care.

Understand that you can have more RAM

Modern boards support ram in at least two different ways and can be accessed from IntyBASIC via the --CC3 or --JLP command line arguments.

If I had remembered that (specifically --CC3) when I initially spliced the 3 games for IntyBASIC Showcase Vol1 together I would have saved myself a crapload of work harmonizing variable names in order to fit into the approximately 193 bytes available on a stock board.

Use Descriptive Variable Names

IntyBASIC doesn't care if you call a variable x or PlayerOneX, but *you* will months from now when you are debugging. That includes loop counters

```
For i=0 to 10
    Gosub Wiggle
Next i
```

instead

```
For LoopCounter=0 to 10
   Gosub Wiggle
Next LoopCounter
```

Use a Code editor

of some kind, not Notepad. There are lots of free ones, I use notepad++ and use the Language-->Visual Basic code highlight setting. You can create your own and someone with more time can create an IntyBASIC one, but the VB one makes life easier. It also does variable autocomplete as you type so you don't have to type (or in some cases remember how you spelled a PROCEDURE name,) just start typing and it will start showing you.

Comment your code

In the case of IntyBASIC Showcase, I had 3 different code sets from 3 different developers, with differing levels of training and development experience, from three different countries (one of which wasn't even in English, that was fun! 🙂)

Even if you are the only one looking at your code, after three months you may not remember what the following does:

```
ZObjPick=RAND/64
IF objtype(ZObjPick)>0 THEN RETURN
```

take a few minutes while you are coding it and do this to some degree

```
'pick an object to fire, 4 possible on screen at any tim
ZObjPick=RAND/64
```

```
IF objtype(ZObjPick)>0 THEN RETURN ' picked an existing
```

Get Help

If you really are going towards a cart release, get help, there is lots out there. Not everyone can code, but there are people can can create sprites, make music, design manuals, boxes etc. You don't have to be a loner. I consider myself a good programmer, with lots of great game ideas, and a good artist. With IntyBASIC Showcase I intended to draw the box cover, but as I was busy working on the coding, my mind would wander to the manual design or the box cover. Finally, after prompting from Rev, I let go of the "I can do everything" mentality, mocked up a cover using some images from the net in Paint.Net that roughly matched what I saw in my head and sent to to a professional artist (you could send to an unprofessional artist 🙂 or an amateur one) Same with the manual. MThompson is great at that, his manuals are awesome! What the helpers want in exchange varies, but explore it.
Zyx now has a large number of unique items as part of the level progression. How many? You have to play it to find out but a lot of them were designed by pimpmaul69 using IntyWORKShop and he posted the BITMAP statements to the private thread, I feel bad that we had to cut a bunch due to space.

Start a private PM thread and hash things out there, even bounce around game ideas. Many of the additions to Zyx and Warship came out of that PM thread.

Test the ever loving shit out of it on as many consoles as you can

We thought we were almost done...until Intvsteve put the near final bin/cfg on his LTO Flash! and the friggening thing constantly but randomly locked up the console, and he tried on 4 different ones (hoarder) with and without the Intellivoice. Thanks to Joe Z, the cause was tracked down to calling VOICE INIT more than once, even tho it works fine in the jzintv emulator, it locks up in certain seemingly random cases on 4 different flavors of consoles. Testing on real hardware aside, get help testing because, as a programmer, you will not do things others might.

As part of that, find some different age groups to test stuff, not just a bunch of middle aged fan boys
Besides the old farts that are testing IntyBASIC Showcase (and the Intellivision Revolution releases) I have the following also play for feedback
2 18 year olds (one boy/one girl)
1 10 year old boy
1 14 year old boy

Their comments and feedback have led to many changes in skill level progression, mechanics and other things. I had to laugh when I had them play a homebrew I bought since I got into buying them 3 years ago and after 10 minutes every single one of them refused to play it ever again...

+Tarzilla
Posted September 18, 2016

Report

Part 2

intvnut, First Spear, Old Timey Retro Gamer and 3 others 6

Call VOICE INIT only once at the top of your program
See above 🙂

Get someone to proof read
See above from 1 to 6 edits ago

Don't be afraid to ask for help from an expert
See next post below 🙂

Use On GOTO or On GOSUB instead of repeated IF/THENs
For instance, don't do the following, especially in a time sensitive loop, you'll lose cycles for every IF that is false, even if you have a GOTO at the end of the IF

```
If level = 1 then GOSUB DrawLevel1 : Goto AfterIFTests
If level = 2 then GOSUB DrawLevel2 : Goto AfterIFTests
If level = 3 then GOSUB DrawLevel3 : Goto AfterIFTests
If level = 4 then GOSUB DrawLevel4 : Goto AfterIFTests
```

```
rem a whole bunch more IFs
If level = 15 then GOSUB DrawLevel15 : Goto AfterIFTests

AfterIfTests:
Rem continue doing stuff
```

instead Do:

```
Rem remember On starts from ZERO not ONE
On (Level-1) Gosub  DrawLevel1,DrawLevel2, DrawLevel3
Rem and so on to DrawLevel15, same line, seperated by co
Rem continue doing stuff
```

Remember Arrays are ZERO indexed

```
Dim EnemyX(5)

For EnemyCounter=0 to 4
   EnemyX(EnemyCounter)=Random(140)+10
Next EnemyCounter
```

Use RANDOM instead of RAND

RAND was the old command, it does not advance the generator, calling it more than once before a WAIT will return the exact same number.

From Kiwi:

Don't Jump out of a Procedure

Otherwise you'll get stack overflow and that will be very bad. To exit the procedure just let it reach return:end and it'll pop back to the program, removing the data from the stack.

EXAMPLE OF NOT TO DO:

tireness=230
gosub overtired
Opps:
overtired:procedure
if tireness>=200 then goto Opps
return
end

YOU CAN DO:
gosub caffeineaddition
caffeineaddition:procedure
tireness=tireness-20
caffeine=caffeine+5
if caffeine>99 then crash=1 goto sleep
sleep:
return
end
Or something like that. You can use GOTO with in a procedure. Just don't use it to jump out of procedure to the main program or another procedure.

Related to the above is:
Don't Gosub too many times/hops away
The return location of a GOSUB is saved on the stack (a special, limited area of memory), when a Procedure ends it pops the return address off the stack, too many consecutive GOSUBs without a RETURN/END and you will get a Stack Error because you filled it.

You don't need a RETURN before an END
RETURN is used to exit a Procedure on demand, and is not necessary before an END

```
DoAmazingStuff: Procedure
Rem do some stuff
AmazingLoop:
    NotAmazing=Random(200)
    If NotAmazing=1 then Return
    If NotAmazing=75 then Gosub CoolStuff : Return
Goto AmazingLoop

Return 'Rem RETURN and END do the same thing in this cas
END
```

Check out Structured Programming
While I don't expect the average hobbyist to take classes, at least learn what the basics are of Structured Programming to limit the use of GOTO's. AVOID GOTOs!!! In most cases structured usage of

GOSUB will keep you organized and thinking in smaller, related tasks/chunks of code.

If you have enough time to pick up IntyBASIC, you have plenty of time to read up on the concepts of Structured Programming. You'll save yourself frustration in the long run, no matter what console or modern programming language you try.
https://en.wikipedia.org/wiki/Structured_programming

As Joe says below, don't waste time IF-ing your way through life, learn the more advanced evaluation and flow control commands.

Use Lookup tables
Lookup Tables are a great way to make stuff happen without wasting a lot of space with IFs or using precious Variables.
He provides an example in post # 11 below, there another example in the INTRO.BAS sample included at no extra charge in every copy of IntyBASIC.
Every screen pauses with a flashing > when waiting for you to proceed. Notice how it isn't randomly flashing and it doesn't just cycle thru all 16 colors?

Look for the **Pause** Procedure, it is used a lot! If you didn't use gosub YOU as the programmer would have to either duplicate the pause loop over and over, or keep track of where to go after GOTOing PAUSE. GOSUB is so much easier. One block of self contained code called as needed, automatically return to where it was called when done. You'll note that there is a GOTO in the Pause procedure. That was written before the WHILE command was added. If INTRO.BAS was written today, there would probably be ZERO GOTOs in the code. Now look for this section immediately following the END (of **Pause**: Procedure)

```
PauseColorArray:
Data CS_DARKGREEN,CS_GREEN,CS_TAN,CS_WHITE,CS_WHITE,CS_T
```

Change some of the values to other colors then Compile and Run it. If you had to repeat the functionality with just IFs, you'd be wasting 8 or more lines on IF/THEN checks.

speaking of INTRO.BAS...

Look at and change the samples included with IntyBASIC

The examples vary in purpose and age, some are from the early releases of IntyBASIC and contain a lot of magic numbers, but you can cross reference CONSTANTS.BAS to see what they do.

INTRO.BAS was written with the beginner in mind to show off the features of IntyBASIC and the Intellivision. Even it should be updated since there have been several releases of IntyBASIC since it was written, too bad the programmer is too busy wondering whether he should enter the *Procrastinator of the Year* awards, (he should also finish Volume 2 which explains Scrolling/Music and other features not covering the the first one.)

The point is:The examples are included to provide a reference, but also for you to play with. Change things and try it. Breaking things is how you learn to fix them.

Added November 9, 2016

The following is not directly related to IntyBASIC but it definitely belongs here before I forget.

Be done when you are done

What does that mean? As a bookend to the **Plan Ahead** entry, in simple terms, don't set yourself artificial deadlines, such as releasing a game (or games) for Portland Retro Gaming Expo 2016!

If you do plan on such a dumb-ass release point for a game (or games,) look in a mirror. See that person staring back? Punch them in the face to knock some sense into them. Unless you have enough lead time to have the completed game tested and sitting dormant for months before the deadline, don't put yourself through the stress of last minute debugging and manufacturing. Shout out to Harvey and Joe for being so accommodating with the last minute weird stuff we were finding but if you want to premiere a game at an Expo that runs in October, be done by the end of July because Real Life will kick you in the nuts when you least expect (or want) it.

Don't overestimate your audience

Related to the item above, for Intellivision games, you will not sell 100 copies at an expo so don't bring that many. For the most part you will be selling 1 or 2 copies (or 10 if Cmart is attending) to each of the INTV cult that is attending and already knows you are going

to be releasing the game there. You won't sell 50 copies to random passers-by, even if you are at the largest Expo in the world.
While the last two statements above may sound a bit negative as a warming, an Expo is a great way to premiere a game and more importantly, meet all the other weirdos that hang out on AtariAge, but temper any expectations you may have as the primary attendees seemed to be Nintendo Entertainment System era and newer. I'd be curious to talk to the NES homebrew vendors that were there to get their thoughts on audience size for their niche.
To be continued:

Edited November 9, 2016 by Tarzilla

nanochess, Gemintronic, DZ-Jay and 2 others 5

Report

intvnut
Posted September 18, 2016

> **On 9/18/2016 at 1:32 PM, Tarzilla said:**
>
> **Part 2**
>
> **Call VOICE INIT only once at the top of your program**

If I could make one feature request for IntyBASIC from all this: In games that use voice, Deprecate `VOICE INIT` and make it NOP. Instead, have the compiler automatically emit a `CALL IV_INIT` (preferably before enabling interrupts) before the first line of code. That way, it's impossible to get that part wrong in an IntyBASIC program.

If IntyBASIC needs a `VOICE SHUTUP` command (or more politely, `VOICE HUSH`), I'd be happy to work one up. It'd basically require flushing the voice queue and queuing up a pause phoneme in a non-interruptible fashion. ie. set `IV.QT` = 0, `IV.QH` = 1, and put a single entry at `IV.Q` to play a 'pause' phoneme (such as `_PA1`) to stop the

Intellivoice from voicing the previous phoneme. The three writes need to happen 'atomically' with respect to the interrupt handler. Three `MVO`s with no intervening instructions would do, or wrapping the call in `DIS`/`EIS` would work too. You don't need to get fancy as it's not performance critical.

Something like this *may* work. I'm not able to test at the moment, though:

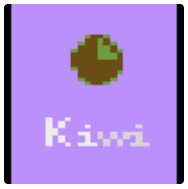
```
IV_HUSH PROC
        CLRR    R0
        MVII    #1, R1
        MVII    #_PA1, R2
        MVO     R0, IV.QT
        MVO     R1, IV.QH
        MVO     R2, IV.Q
        JR      R5
        ENDP
```

You don't want to modify `IV.FPTR` or `IV.FLEN`, as you want to let the currently speaking phoneme fully enter the queue, so you remain in sync with the microsequencer.

EDIT: Rather than _PA1 above, you actually want the #5, which is RESROM.pa1. That's a 50ms pause. I forgot the IntyBASIC driver does keep the phrase pointer (IV.PPTR), it just eliminates the phrase table. It still checks for phrase number < 43 to know when to play something from RESROM, and still has the logic to interleave RESROM samples w/ user phrases. I don't think IntyBASIC exposes the RESROM phrases though. Correct me if I'm wrong.

Edited September 18, 2016 by intvnut

Quote First Spear and nanochess 2



Kiwi
Posted September 18, 2016

Report

DON'T JUMP OUT OF PROCEDURE
Otherwise you'll get stack overflow and that will be very bad. To exit the procedure just let it reach return:end and it'll pop back to the program, removing the data from the stack.

EXAMPLE OF NOT TO DO:

```
tireness=230
gosub overtired
Opps:

overtired:procedure
if tireness>=200 then goto Opps
return
end
```

YOU CAN DO:

```
gosub caffeineaddition

caffeineaddition:procedure
tireness=tireness-20
caffeine=caffeine+5
if caffeine>99 then crash=1 goto sleep
sleep:
return
end
```

Or something like that. You can use goto with in a procedure. Just don't use it to jump out of procedure to the main program or another procedure.

Ok I got that covered I think.

Quote nanochess, DZ-Jay, intvnut and 1 other 4

Report

intvnut
Posted September 18, 2016

> **On 9/18/2016 at 8:09 PM, Kiwi said:**
>
> Just don't use it to jump out of procedure to the main program or another procedure.

There *are* ways to make it work (involving some inline ASM). But unless you *really* know what you're doing, you're better off not knowing them. And even if you do know what you're doing, you're best off avoiding them. GOTO in general tends to lead to unmaintainable spaghetti. It's best used judiciously, such as for exiting through a common point, or early-exiting a loop.

One thing I always found lacking in most (but not all!) BASICs is a sense of 'scope'. Scope prevents you from this sort of 'goto' across procedures in other languages, such as C and Pascal. (Or TI Extended BASIC, IIRC.)

Scope also allows for other nice features, like local variables that can share the same memory, and proper recursive functions. Maybe someday in an IntyBASIC 2.0.

Quote

Report

+Tarzilla
Posted September 18, 2016

> **On 9/18/2016 at 8:09 PM, Kiwi said:**
>
> **DON'T JUMP OUT OF PROCEDURE**
> Otherwise you'll get stack overflow and that will be very bad. To exit the procedure just let it reach return:end and it'll pop back

to the program, removing the data from the stack.

EXAMPLE OF NOT TO DO:

```
tireness=230
gosub overtired
Opps:

overtired:procedure
if tireness>=200 then goto Opps
return
end
```

YOU CAN DO:

```
gosub caffeineaddition

caffeineaddition:procedure
tireness=tireness-20
caffeine=caffeine+5
if caffeine>99 then crash=1 goto sleep
sleep:
return
end
```

Or something like that. You can use goto with in a procedure. Just don't use it to jump out of procedure to the main program or another procedure.

Ok I got that covered I think.

I've never done that but very good note. Consider it ~~stolen~~ promoted.

Quote

Kiwi and DZ-Jay 2

Report

Second Post updated with more Do's and Don'ts

+Tarzilla
Posted September 18, 2016

Quote

Gemintronic 1

Report

On 9/18/2016 at 8:56 PM, intvnut said:

There *are* ways to make it work (involving some inline ASM). But unless you *really* know what you're doing, you're better off not knowing them.

+DZ-Jay
Posted September 19, 2016

Too bad you just mentioned it. DOH! 😄

I think we can agree that a "DON'T" rule of "***AVOID GOTOs!!!*** *(unless you really have a good reason for it)*" should be painted in red bold letters. 🙂 (Of course there are exceptions like "ON GOTO," but generally we should promote structured programming with GOSUB.)

I believe IntyBASIC has reached a point of maturity where an entire game program can be constructed without resorting to flow control tricks with GOTO. Sub-routines (GOSUB) should be emphasized.

-dZ.

Quote

+Tarzilla
Posted September 19, 2016

Report

> **On 9/19/2016 at 7:20 AM, DZ-Jay said:**
>
> Too bad you just mentioned it. DOH! 😄
>
> I think we can agree that a "DON'T" rule of "***AVOID GOTOs!!!*** *(unless you really have a good reason for it)*" should be painted in red bold letters. 🙂 (Of course there are exceptions like "ON GOTO," but generally we should promote structured programming with GOSUB.)
>
> I believe IntyBASIC has reached a point of maturity where an entire game program can be constructed without resorting to flow control tricks with GOTO. Sub-routines (GOSUB) should be emphasized.
>
> -dZ.

I'm confused...did you mean?
DON'T AVOID GOTOs!!!

😊

Quote

+DZ-Jay
Posted September 19, 2016

Report

> **On 9/19/2016 at 9:58 AM, Tarzilla said:**
>
> I'm confused...did you mean?
> **DON'T AVOID GOTOs!!!**

DOH!

I thought it would be weird to include a "negative" rule in the "DO's" (i.e., "Do avoid GOTOs'"), but yeah, I can see that it sounded silly.

For everyone reading this, to be sure, the rule should be:

Avoid GOTOs!!!

Quote

Report

intvnut
Posted September 19, 2016

This is a really good tip: **Use On GOTO or On GOSUB instead of repeated IF/THENs**

There is a related tip too: If you're just setting a couple variables based on the value of another (for example, changing the color or position of something) with a stack of IF-THENs, a lookup table is a better choice.

That is, instead of:

```
IF Level = 1 THEN Color = CS_RED : X = 42 : Y = 17
IF Level = 2 THEN Color = CS_BLUE : X = 51 : Y = 13
IF Level = 3 THEN Color = CS_GREEN : X = 39 : Y = 19
IF Level = 4 THEN Color = CS_YELLOW : X = 40 : Y = 15
```

You can instead say:

```
Color = LevelColors( Level )
X = LevelXPos( Level )
```

```
Y = LevelXPos( Level )

' ... and elsewhere, outside of any procedure, have:

LevelColors:    DATA  CS_RED, CS_BLUE, CS_GREEN, CS_YELL
LevelXPos:      DATA  42, 51, 39, 40
LevelYPos:      DATA  17, 13, 19, 15
```

In general, if you find yourself with a stack of IF-THENs, there are often (but not always!) better ways to tackle it.

Quote nanochess, carlsson, First Spear and 1 other 4

Report

+Tarzilla
Posted September 19, 2016

Added more to the bottom of Post #2

Quote nanochess and intvnut 2

Report

+DZ-Jay
Posted September 19, 2016

> **On 9/19/2016 at 2:20 PM, Tarzilla said:**
>
> Added more to the bottom of Post #2

Quote

1 month later...

Report

Added some slightly off-topic thoughts to Post #2

+Tarzilla
Posted November 9, 2016

Quote

nanochess 1

Reply to this topic...

Share Follow 1

Go to topic listing

Next unread topic

Recently Browsing 1 Member

paisleyboxers

Home > Classic Consoles > Intellivision / Aquarius > Intellivision Programming > IntyBASIC Tips/Tricks and Do's and Don'ts

Theme Privacy Policy Contact Us Cookies

Powered by Invision Community

AtariAge

Home
2600
5200
7800
Lynx

Forums

Forum Home
Clubs
Blogs
Gallery
Subscriptions

Jaguar

Store

Store Home
New Products
Bestsellers
Gift Certificates

General

Contact
Submit News
Privacy Policy
Legal

Follow AtariAge

Like us on Facebook
AtariAge Facebook Group
Follow us on Twitter
AtariAge on Instagram
Watch us on YouTube
Chat on Discord

Copyright ©1998-2023 AtariAge